TRAITEMENT DE DONNÉES · FICHE DE RÉVISION · TD 1

Descente de gradient

Polynôme convexe, nombre d'or, gradient à pas fixe, critères d'arrêt, cas multi-variable, pas optimal

À RETENIR EN PRIORITÉ

- **Mise à jour** : $x_{t+1} = x_t - \mu \cdot \nabla f(x_t)$ — on va **à l'opposé du gradient** (le gradient pointe vers la montée). μ = **pas** (hyperparamètre, choisi à la main).
- **Recalculer le gradient au nouveau point** à chaque itération.
- **Pas trop petit** → convergence très lente. **Pas trop grand** → oscillations, puis **divergence**. Ici ($f = 2x^2 - 4x + 5$) : converge ⇔ **$0 < \mu < 0{,}5$**.
- **Critères d'arrêt** : $\|\nabla f(x)\| \leq \varepsilon$ (convergence) · nombre max d'itérations · f ne diminue plus (divergence détectée).
- Multi-variable : $f(\mathbf{x}) = \mathbf{x}^T A\mathbf{x} + \mathbf{x}^T\mathbf{b} + c \Rightarrow \nabla f = (A + A^T)\mathbf{x} + \mathbf{b}$ (pas $2A\mathbf{x}$ si A n'est pas symétrique).
- **Pas optimal** : à chaque itération, $\mu_k = \text{argmin}\, g(\mu) = f(\mathbf{x}_k - \mu\nabla f(\mathbf{x}_k))$, trouvé par la **méthode du nombre d'or** → 12 itérations au lieu de 144.
- **Convexe** ⇒ un seul minimum : x_0 ne change que le nombre d'itérations. **Non convexe** ⇒ risque de minimum local, résultat dépendant de x_0.

1. Le problème : trouver un minimum

Au cours 1, apprendre un modèle = trouver $\boldsymbol{\mu^* = \text{argmin}_\mu\, g(f_{\mu,\lambda}, \{X, Y\})}$. En général pas de formule fermée, fonction parfois non convexe, beaucoup de paramètres → il faut une **méthode itérative** : c'est l'objet du TD.

Polynôme d'ordre 2 : $f(x) = ax^2 + bx + c$

- **Convexe** ⇔ $f''(x) = 2a > 0$ ⇔ **$a > 0$** (b et c quelconques).
- Minimum où $f'(x) = 2ax + b = 0 \Rightarrow$ **$x_{opt} = -b / (2a)$**.

Le polynôme du TD : a = 2, b = −4, c = 5
$f(x) = 2x^2 - 4x + 5$ · **$f'(x) = 4x - 4$** (`compute_df`)
$x_{opt} = 4 / 4 =$ **1** · $f(1) = 2 - 4 + 5 =$ **3** (et non c = 5)

PIÈGE DE NOTATION

Dans le **cours**, μ désigne les **paramètres du modèle** (ce qu'on cherche). Dans le **TD**, μ est le **pas** de la descente (souvent noté η ou « learning rate » ailleurs). Le pas est un **hyperparamètre** : il n'est pas appris.

2. Méthode du nombre d'or (fonction à une variable)

Conditions : f **mono-variable** et **unimodale** sur un intervalle (x_1, x_3) qui **contient le minimum**. Aucune dérivée, aucun pas : seulement des valeurs de f.

Sondes ($\phi = (1 + \sqrt{5})/2 \approx 1{,}618$; $1/\phi \approx 0{,}618$) :

$$\boldsymbol{x_2 = x_3 - (x_3 - x_1)/\phi \quad \cdot \quad x_4 = x_1 + (x_3 - x_1)/\phi}$$

- Si **$f(x_2) < f(x_4)$** : minimum dans (x_1, x_4) → `x_3 = x_4`.
- Sinon : minimum dans (x_2, x_3) → `x_1 = x_2`.
- Arrêt quand **$|x_2 - x_4| \leq \varepsilon$** ; on renvoie $(x_1 + x_3)/2$.
- L'intervalle est multiplié par **0,618** à chaque itération (comme une dichotomie, mais avec ϕ au lieu de 2).

EXEMPLE DU TD : INTERVALLE (−100 ; 100)

```
(x3 - x1)/φ = 200 / 1,618 ≈ 123,6
x2 = 100 - 123,6  = -23,6   f(x2) ≈ 1214
x4 = -100 + 123,6 = +23,6   f(x4) ≈ 1025
f(x2) > f(x4)  →  x1 = x2
nouvel intervalle : (-23,6 ; 100)
nouvelles sondes  : 23,6 et 52,8
```

Sortie de la correction ($\varepsilon = 10^{-10}$) : `x_opt = 1.0000000104692428`.

3. Algorithme du gradient

Il ne demande qu'un **point de départ x_0** (et un pas) et marche en dimension quelconque. Le nombre d'itérations n'est pas connu d'avance.

MÉTHODE : FAIRE UN PAS DE DESCENTE DE GRADIENT À LA MAIN

1. **Dériver** f (ou calculer le vecteur gradient en multi-variable).
2. **Évaluer** le gradient au point courant x_t.
3. **Multiplier par le pas** μ.
4. **Soustraire** : $x_{t+1} = x_t - \mu \cdot f'(x_t)$. Gradient négatif ⇒ x augmente.
5. **Contrôler** : $f(x_{t+1}) < f(x_t)$? Sinon le pas est trop grand (ou il y a une erreur de signe).
6. Pour le pas suivant, **recalculer le gradient en x_{t+1}**.

1D : $f = 2x^2 - 4x + 5$, $x_0 = 10$, $\mu = 0{,}1$

```
f'(10)   = 4×10 − 4    = 36
x1 = 10  − 0,1×36      = 6,4     f: 165 → 61,32
f'(6,4) = 4×6,4 − 4    = 21,6
x2 = 6,4 − 0,1×21,6    = 4,24    f: → 23,995
```

2D : f du TD, $\mathbf{x}_0 = (-5, -5)$, $\mu = 0{,}05$

```
A + Aᵀ = [4 3 ; 3 8]
∇f(x0) = (−35, −55) + (1, 2) = (−34, −53)
x1 = (−5, −5) − 0,05×(−34, −53)
   = (−5 + 1,7 ; −5 + 2,65)
   = (−3,3 ; −2,35)          f: 212 → 61,135
```

ERREURS TYPIQUES SUR UN PAS

- **Signe +** : 10 + 3,6 = 13,6 → on monte.
- **Réutiliser l'ancien gradient** : 6,4 − 0,1×36 = 2,8 au lieu de 4,24.
- **Utiliser f au lieu de f'** : 10 − 0,1×165 = −6,5.
- En 2D : prendre **2Ax + b** (→ (−29, −58)) ou **oublier b** (→ (−35, −55)).

4. Influence du pas μ (pas fixe)

Pour $f = 2x^2 - 4x + 5$: $x_{t+1} - 1 = \mathbf{(1 - 4\mu)}(x_t - 1)$. La distance au minimum est multipliée par $1 - 4\mu$ à chaque pas ⇒ convergence ⇔ $|1 - 4\mu| < 1$ ⇔ **$0 < \mu < 0{,}5$**. Départ $x_0 = 10$ (cellule de la correction) :

μ	$1 - 4\mu$	x_1	x_2	x_{10}	Comportement	`algo_gradient2` ($\varepsilon = 10^{-10}$)
0,0001	0,9996	9,9964	9,9928	9,964	**Trop lent** : x a à peine bougé	41 672 itérations
0,1	0,6	6,4	4,24	1,054	Converge sans osciller	42 itérations
0,25	0	1	1	1	Minimum en **un pas** ($\mu = 1/f''$)	1 itération
0,4	−0,6	−4,4	4,24	1,054	Converge en **oscillant** autour de 1	42 itérations
0,5	−1	−8	10	10	Oscille **sans fin** entre 10 et −8	arrêt après 1 pas : f(−8) = f(10) = 165
0,6	−1,4	−11,6	18,64	261,3	**Diverge** en oscillant	arrêt après 1 pas (f augmente)

PIÈGES SUR LE PAS (LEARNING RATE)

- **Plus grand n'est pas plus rapide** : μ = 0,1 et μ = 0,4 demandent toutes deux 42 itérations ; au-delà de 0,5 plus rien ne converge.
- **Osciller ≠ diverger** : μ = 0,4 oscille mais converge ; μ = 0,5 oscille sans converger ; μ = 0,6 diverge.
- **Trop petit ne « casse » rien** mais coûte énormément d'itérations (et peut buter sur `max_iter`).
- La plage de bons pas **dépend de la fonction** (de sa courbure), pas du point de départ. En 2D (essais hors TD, $x_0 = (-5, -5)$) : μ = 0,05 → 144 itérations, 0,1 → 70, 0,2 → 244, 0,21 → divergence dès le 1er pas.

5. Critères d'arrêt et lecture du code

```
def algo_gradient3(f, df, mu, x_0, epsilon, max_iter):
    x = x_0; x_hist = [x]
    fx = f(x); dfx = df(x); k = 0
    while(np.linalg.norm(dfx) > epsilon
          and k < max_iter):
        x = x - mu*dfx           # pas de descente
        x_hist.append(x)
        if(f(x) >= fx):          # f ne baisse plus
            break                # → divergence
        fx = f(x); dfx = df(x)   # gradient recalculé
        k = k + 1
    return x, x_hist
```

Test	Rôle
$\|\nabla f(x)\| \le \varepsilon$ (1D : $\|f'(x)\| \le \varepsilon$)	Convergence : gradient quasi nul = minimum
k < max_iter	Garantit que la boucle se termine (convergence lente, ε inatteignable)
f(x) ≥ fx → break	Détecte la divergence ; renvoie quand même le dernier point (pour le tracer)
`n` fixé (`algo_gradient1`)	Pas de critère : n itérations, renvoie l'historique (`x_hist[-1]` = résultat)

LIRE LES SORTIES

Avec μ = 0,05, la correction affiche `compute_df2(x_opt_2) = [-4.6e-08 2.5e-08]` : la norme est **au-dessus** de ε = 10^{-10} et seules 144 itérations ont tourné. C'est donc le test **f(x) ≥ fx** qui a arrêté la boucle : f ne baisse plus à la précision machine près. Ce n'est pas une divergence, x est bien à l'optimum (−0,087 ; −0,217).

6. Cas multi-variable

Même règle, en vecteurs : $\mathbf{x_{k+1} = x_k - \mu \cdot \nabla f(x_k)}$

$f(\mathbf{x}) = \mathbf{x}^T A\mathbf{x} + \mathbf{x}^T\mathbf{b} + c$, avec A = [2 1 ; 2 4] (**non symétrique**), **b** = (1, 2), c = 2.

- **$\nabla f(\mathbf{x}) = (A + A^T)\mathbf{x} + \mathbf{b}$**, avec $A + A^T$ = [4 3 ; 3 8].
- Minimum : $\nabla f = 0 \Leftrightarrow \mathbf{(A + A^T)x = -b} \Rightarrow \mathbf{x}_{opt} = -(1/23)(2, 5) \approx (-0{,}087 ; -0{,}217)$, $f(\mathbf{x}_{opt}) = 40/23 \approx 1{,}739$.
- `x` doit être un vecteur numpy **1D de forme (2,)** : `np.array((-5., -5.))`, `np.zeros(2)`. Pas `np.zeros((2,1))` (le gradient devient une matrice 2×2), pas une liste (pas de `.T`).

Valeurs en $\mathbf{x}_0 = (-5, -5)$

```
xᵀAx = 25×(2+1+2+4) = 225
xᵀb  = −5 − 10      = −15
f(x0) = 225 − 15 + 2 = 212
∇f(x0) = (−34, −53)
```

Erreurs : 242 (signe de $\mathbf{x}^T\mathbf{b}$), 210 (oubli de c), 137 (diagonale seule).

7. Pas optimal

Le pas fixe demande beaucoup d'itérations et un bon μ est difficile à trouver. Idée : une fois la direction $-\nabla f(\mathbf{x}_k)$ connue, chercher **jusqu'où descendre** :

$$\mu_k = \text{argmin}_{\mu>0}\, g(\mu), \quad g(\mu) = f(\mathbf{x}_k - \mu \cdot \nabla f(\mathbf{x}_k))$$

```
ft = lambda mu : f(x - mu*dfx)    # g(μ), mono-variable
mu_opt = mno(ft, 0, 100, 1e-10)  # nombre d'or
x = x - mu_opt*dfx
```

	Pas fixe μ = 0,05	Pas optimal
Itérations	144	**12**
Réglage de μ	à la main	aucun
Coût d'une itération	1 gradient	+ une recherche du nombre d'or (≈ 55 tours)
Résultat	même optimum, f ≈ 1,7391 (fonction convexe)	

1er pas optimal depuis (−5, −5) : $\mu_0 \approx 0{,}105$. En pratique, `scipy.optimize.minimize(compute_f2, x_0, jac=compute_df2, method='CG')` donne directement (−0,0870 ; −0,2174).

8. Convexité, minimum local et initialisation

	Fonction convexe (cas du TD)	Fonction non convexe
Minimums	Un seul : local = global	Plusieurs minimums locaux possibles
Gradient nul	⇒ minimum global	⇒ seulement un point critique (minimum local…)
Rôle de x_0	Change seulement le **nombre d'itérations**	Change aussi le **minimum atteint**
Règle de mise à jour	Identique : $x \leftarrow x - \mu\nabla f(x)$. Seule la garantie d'optimum global disparaît.	